JAVA SDE-2 INTERVIEW PREPARATION SERIES

VOLUME 14 OF 18

# Heaps, Priority Queues, Selection, and Top-K

## Heap Invariants, Comparator Safety, K-Way Processing, and Partial Ordering


FOUNDING AUTHOR

**Vinay Reddy Kalluri**

EDITOR-IN-CHIEF | CHIEF AUDITOR


Use heap order when only the next extremum matters, compare full sorting with selection, and write stable, overflow-safe Java priority logic.

HEAPS | PRIORITYQUEUE | TOP-K | SELECTION | COMPARATORS | MERGING

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION


Series edition - Release 2026-07-25

Open educational interview-preparation edition

# Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `13 – Trees, BSTs, and Tries`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This volume distinguishes heap order from global sorting and develops priority-based interview patterns without hiding comparator or mutation risks.

## Readiness map

| Decision | Evidence |
|---|---|
| **START HERE** | Only the next minimum or maximum is needed; The best k items must be retained; Several sorted streams must be merged; Scheduling decisions are priority-driven. |
| **PREPARE FIRST** | Volumes 1-13; Arrays, trees, sorting, and comparators. |
| **MOVE ON WHEN** | Explain array-backed heap invariants; Use PriorityQueue safely; Choose between sorting, selection, and a bounded heap; Avoid mutable-priority and comparator failures. |

## Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

## Volume Position

| 13 - Trees, BSTs, and Tries | YOU ARE HERE<br>**14 - Heaps and Priority Queues** | 15 - Graphs |
|---|---|---|

# Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

# Chapter 1 - SDE-2 Heaps, Priority Queues, Selection, and Top-K Patterns

## Why this chapter matters

A heap answers one focused question efficiently: which current item has the highest priority? That makes it the right structure for bounded top-k state, merging sorted sources, streaming medians, and event scheduling. It is not a fully sorted collection, a search tree, or a general indexing structure.

At SDE-2 level, "use a priority queue" is only the beginning. A complete answer defines the comparator, proves the heap or bounded-candidate invariant, handles ties and overflow, accounts for retained state, and explains deletion, streaming, and alternative selection strategies.

## Learning objectives

After completing this chapter, you should be able to:

- derive parent and child indexes and state the heap-order and shape invariants;
- use Java `PriorityQueue` without comparator, mutation, or null mistakes;
- build a heap in linear time with bottom-up heapify;
- solve kth-largest and top-k-frequency problems with bounded heaps;
- merge k sorted sources while storing only one frontier item per source;
- maintain a streaming median with two balanced heaps;
- model meeting rooms and event simulation as scheduling heaps;
- explain lazy deletion when arbitrary removal is otherwise linear;
- compare a heap with sorting, counting, and quickselect; and
- discuss capacity, determinism, concurrency, backpressure, and numeric safety.

## CHOOSE IT | Recognition and decision map

| Signal | Candidate technique | Retained state |
|---|---|---|
| repeatedly remove current minimum or maximum | priority queue | all active candidates |
| kth largest, stream or one pass | min-heap of size k | best k values |
| top k by frequency | frequency map plus bounded heap | all counts, then k candidates |
| merge k sorted sources | min-heap frontier | next item from each nonempty source |
| median after every insertion | max-heap lower half plus min-heap upper half | every value, split by rank |
| minimum simultaneous resources | min-heap of finishing times | active resource end times |
| one kth element in a mutable array | quickselect | in-place partition state |
| complete sorted output | sorting | entire input |
| dense, small priority range | bucket/counting structure | counts per priority |

A heap is compelling when only an extreme matters after each update. If every result must be sorted, sorting once is often simpler. If values come from a tiny bounded domain, a counting array may beat a node-heavy queue.

## WHY IT WORKS | Heap model and invariants

A binary heap is a complete binary tree stored level by level in an array. Completeness supplies the shape invariant: every level except possibly the last is full, and the last fills from left to right. For zero-based index `i`:

TEXT

```
parent(i) = (i - 1) / 2, for i > 0
left(i)   = 2 * i + 1
right(i)  = 2 * i + 2
```

In a min-heap, every parent is no greater than either child. The root is therefore a global minimum. Siblings and separate subtrees have no ordering relationship. Iterating a Java priority queue does not produce sorted order.

Insertion appends at the complete-tree frontier, then sifts upward while heap order is violated. Removing the root moves the last item to index zero, reduces the shape, then sifts downward toward the better child. Height is `O(log n)`, so insertion and root removal are `O(log n)`; root inspection is `O(1)`.

The sift-down invariant is that only the current node may violate heap order; both child subtrees are already heaps. Swapping with the larger child for a max-heap repairs the old level and moves the possible violation downward. The remaining height strictly decreases.

## Java PriorityQueue contracts

`PriorityQueue<E>` is a min-priority queue according to natural order or a supplied comparator. A max-heap uses a reversed comparator:

JAVA

```
PriorityQueue<Integer> maximums =
        new PriorityQueue<>(Comparator.reverseOrder());
```

Use `Integer.compare(a, b)` or comparator factories, not `a - b`, because subtraction can overflow and reverse ordering. Define every tie needed for deterministic output. A comparator must be antisymmetric, transitive, and consistent enough to establish an ordering; a broken comparator can produce inexplicable queue behavior.

The queue rejects null. `peek()` and `poll()` return null when empty, while `element()` and `remove()` throw. Choose deliberately. Removing an arbitrary object with `remove(Object)` is linear because the heap does not index its internal position.

Never mutate fields used by the comparator while an object is inside the queue. The queue does not notice the priority changed and does not relocate the object. Insert an immutable snapshot or remove and reinsert through a controlled index-aware design.

## Pattern 1: bottom-up heapify

To turn an arbitrary array into a max-heap, every leaf is already a one-node heap. Starting at the last parent `n / 2 - 1`, sift each node down toward index zero.

Invariant before processing index `i`: every subtree rooted at an index greater than `i` is already a max-heap. Sifting `i` repairs its subtree because its child subtrees satisfy the invariant. At termination, index zero roots a heap spanning the array.

Although an individual sift can take `O(log n)`, bottom-up heapify is `O(n)`. Most nodes are near the leaves and move at most one or two levels. Summing `nodesAtHeight(h) * h` over heights is bounded by a constant multiple of `n`.

### TRACE IT | Dry run

For `[3,1,6,5,2,4]`, the last parent is index 2. Index 2 already dominates child 4. At index 1, swap 1 with child 5: `[3,5,6,1,2,4]`. At index 0, swap 3 with 6, then with 4: `[6,5,4,1,2,3]`. Every parent now dominates its children.

Heapify is useful for heap sort or an internal primitive heap. Constructing Java `PriorityQueue` from a collection also performs implementation-managed heap construction; prefer the library unless the interview asks for derivation or primitive storage matters.

## Pattern 2: kth largest with a bounded min-heap

Keep a min-heap containing the largest `k` values seen. For each value:

1. add it;
2. if size exceeds `k`, remove the minimum.

Invariant after each prefix: the heap contains exactly the largest `min(k, processed)` values from that prefix, including duplicates according to occurrence. The root is the smallest among those retained values. After all values, it is the kth largest.

For `[3,2,1,5,6,4]`, `k=2`, retained heaps by sorted content are `[3]`, `[2,3]`, `[2,3]`, `[3,5]`, `[5,6]`, `[5,6]`; the root 5 is second largest.

Time is `O(n log k)`, space `O(k)`. Validate `1 <= k <= n`. Sorting takes `O(n log n)` and may be preferable when all ranks are later needed. Quickselect has expected `O(n)` for one offline rank but mutates the input and has a quadratic worst case without stronger pivot selection.

## Pattern 3: top k frequent values

First count every value in a hash map. Then keep a size-k min-heap of frequency records. The root represents the least desirable retained record, so it is the first evicted when a stronger candidate arrives.

Tie policy is part of the answer. The implementation defines higher frequency as better and, for equal frequency, smaller numeric value as better. The eviction comparator therefore treats lower frequency as worse and, on ties, larger value as worse. Final results are sorted into presentation order.

For values `[1,1,1,2,2,3]`, `k=2`, counts are `{1=3,2=2,3=1}`. The bounded heap retains 1 and 2. Expected time is `O(n + d log k)` for `d` distinct values, and space is `O(d + k)`. Bucket-by-frequency can reach `O(n)` time but allocates buckets indexed up to `n`; sorting the `d` entries costs `O(d log d)` and may be simpler for large `k`.

## Pattern 4: k-way merge

Each sorted source exposes only its next unconsumed value. Put one cursor per nonempty source in a min-heap. Repeatedly remove the smallest cursor, emit its value, and insert the next value from that same source.

Invariant: the heap contains exactly the first unconsumed value of each nonexhausted source. Because each source is sorted, the smallest unconsumed value globally must be among those frontier items. Emitting the heap root is therefore safe.

For `[1,4,9]`, `[2,2,8]`, and `[3,7]`, the initial frontier is 1,2,3. Emitting 1 exposes 4; emitting the first 2 exposes another 2; and so on. Output is `[1,2,2,3,4,7,8,9]`.

For `N` total elements and `k` sources, time is `O(N log k)` and heap space is `O(k)`, excluding output. A streaming version need not retain the entire result. It should close sources, handle one failed source according to policy, and apply backpressure when consumers are slow.

## Pattern 5: streaming median with two heaps

Maintain:

- `lower`: a max-heap containing the lower half;
- `upper`: a min-heap containing the upper half.

Invariants:

1. every value in `lower` is no greater than every value in `upper`;
2. their sizes differ by at most one; and
3. `lower` has the extra item when the total count is odd.

Insert into `lower` when the value is at most its root, otherwise into `upper`. Rebalance by moving a root across if sizes violate the rule. Median is `lower.peek()` for odd count and the average of both roots for even count.

For stream `5,2,10,4`, heap contents by sorted meaning become: lower `[5]`; lower `[2]`, upper `[5]`; lower `[5,2]`, upper `[10]`; lower `[4,2]`, upper `[5,10]`. Medians are `5,3.5,5,4.5`.

Each insertion costs `O(log n)` and median lookup `O(1)`; space is `O(n)`. Compute the even median as `((long) a + b) / 2.0` to avoid integer overflow.

## Pattern 6: scheduling with finishing-time heaps

For minimum meeting rooms, sort intervals by start. Keep a min-heap of end times for meetings currently occupying rooms. Before adding a meeting, remove every end time no greater than its start under half-open interval semantics `[start, end)`. A zero-length interval `[t,t)` is empty and consumes no room, so skip it. Add the end of each nonempty meeting and record the maximum heap size.

Invariant before adding the next sorted meeting: the heap contains exactly the end times of earlier meetings overlapping its start. Its size is the resources currently busy. Adding the meeting gives current concurrent demand.

For `[0,30)`, `[5,10)`, `[15,20)`, maximum size is 2. The meeting ending at 10 is removed before 15 begins, so its room is reused.

Time is `O(n log n)` for sorting and heap operations; space is `O(n)`. If only room count is needed, sorting separate start and end arrays enables a two-pointer sweep. A heap is more natural when assigning room identifiers or processing an online event stream.

The same model supports CPU/job scheduling, timers, and simulation: priority is the next event time. Real schedulers add cancellation, priority classes, fairness, and starvation prevention, which a single comparator does not solve automatically.

## Lazy deletion for arbitrary removal

Priority queues efficiently remove only the root. Sliding-window median and cancellable jobs may need to remove an item that is buried inside. Calling `remove(Object)` for every expiry can make the algorithm quadratic.

Lazy deletion separates logical deletion from physical removal:

1. give items stable identities or use value counts when duplicates are interchangeable;
2. record cancelled/expired identities in a delayed-deletion map;
3. decrement logical size immediately;
4. whenever inspecting a heap root, repeatedly discard roots marked delayed and decrement their delayed counts.

Invariant: logical sizes exclude delayed items even if their nodes remain physically present; after pruning, a visible root is live. This supports amortized `O(log n)` updates because each stale node is physically removed once.

The design is subtle with duplicate values and two heaps. Value-only deletion must know which logical side loses size; stable `(value,id)` entries avoid ambiguity. Delayed maps can retain metadata if pruning never reaches buried cancelled items, so periodic rebuild or queue compaction may be needed in long-running systems.

## Quickselect boundary

Quickselect partitions around a pivot: values less than the pivot move to one side and greater values to the other. Only the partition containing the target rank is explored. Expected time is `O(n)`, auxiliary stack space is expected `O(log n)` or `O(1)` iteratively, and the input is mutated. Poor pivots can cause `O(n^2)` time; randomized pivots make that unlikely but not impossible.

Choose quickselect for one or a few offline ranks when mutation is acceptable. Choose a bounded heap for streaming data, immutable input, small `k`, or ongoing updates. Choose sorting when ordered output is required. A deterministic linear selection algorithm exists, but its constants and implementation complexity rarely make it the first production choice.

## Testing heap algorithms beyond examples

Example assertions catch boundary mistakes, but heap code benefits from properties that hold across generated inputs. After heapify, check every parent against both existing children rather than comparing only the root. Also verify the output array is a permutation of the input; heap order alone would not reveal a duplicated or lost value. For repeated removal, emitted values must be monotone and their multiset must equal the input multiset.

For kth selection, compare random small cases with a sorted reference and test every valid `k`, including duplicates and `Integer.MIN_VALUE/MAX_VALUE`. For top-k frequency, independently count the output and ensure no excluded item is better than a retained item under the complete tie comparator. For k-way

merge, verify global sortedness, output count, and exact multiplicity from every source. Empty sources, one source, all-equal values, and highly uneven source sizes expose frontier errors.

The two-heap median has three useful properties after every insertion: size difference is at most one, lower root is no greater than upper root, and the reported value matches a sorted reference for a short test stream. Test even medians at integer extremes to catch overflow. Scheduling tests should include zero-duration intervals, many simultaneous starts, and exact end/start ties under the chosen half-open contract.

For a custom heap, randomized operation sequences can be checked against `PriorityQueue`. Record a seed when a test fails so the sequence is reproducible. Avoid tests that assume `PriorityQueue` iteration order; compare repeated polls from a copy when sorted priority order is required. These properties turn an implementation invariant into an executable release gate.

## Runnable Java 21 reference implementation

Run with `java -ea HeapSelectionPatterns`.

JAVA (continued 1/7)

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;

public final class HeapSelectionPatterns {
    private HeapSelectionPatterns() {
    }

    private record Frequency(int value, int count) {
    }

    private record Cursor(int value, int source, int index) {
    }

    public static void heapifyMax(int[] values) {
        requireArray(values);
        for (int i = values.length / 2 - 1; i >= 0; i--) {
            siftDownMax(values, i, values.length);
        }
    }

    public static boolean isMaxHeap(int[] values) {
        requireArray(values);
        for (int parent = 0; parent < values.length / 2; parent++) {
            int left = 2 * parent + 1;
            int right = left + 1;
            if (values[parent] < values[left]
                    || right < values.length && values[parent] < values[right]) {
                return false;
```

JAVA (continued 2/7)

```
            }
        }
        return true;
    }

    public static int kthLargest(int[] values, int k) {
        requireArray(values);
        if (k < 1 || k > values.length) {
            throw new IllegalArgumentException("k must be in 1..n");
        }
        PriorityQueue<Integer> retained = new PriorityQueue<>();
        for (int value : values) {
            retained.add(value);
            if (retained.size() > k) {
                retained.remove();
            }
        }
        return retained.element();
    }

    public static List<Integer> topKFrequent(int[] values, int k) {
        requireArray(values);
        Map<Integer, Integer> counts = new HashMap<>();
        for (int value : values) {
            counts.merge(value, 1, Integer::sum);
        }
        if (k < 0 || k > counts.size()) {
            throw new IllegalArgumentException("k must be in 0..distinct");
        }
        Comparator<Frequency> worstFirst = Comparator
                .comparingInt(Frequency::count)
                .thenComparing(Comparator.comparingInt(Frequency::value).reversed());
        PriorityQueue<Frequency> best = new PriorityQueue<>(worstFirst);
```

JAVA (continued 3/7)

```
        for (Map.Entry<Integer, Integer> entry : counts.entrySet()) {
            best.add(new Frequency(entry.getKey(), entry.getValue()));
            if (best.size() > k) {
                best.remove();
            }
        }
        List<Frequency> ordered = new ArrayList<>(best);
        ordered.sort(Comparator.comparingInt(Frequency::count).reversed()
                .thenComparingInt(Frequency::value));
        return ordered.stream().map(Frequency::value).toList();
    }

    public static List<Integer> mergeSorted(int[][] sources) {
        if (sources == null) {
            throw new IllegalArgumentException("sources must not be null");
        }
        PriorityQueue<Cursor> frontier = new PriorityQueue<>(
                Comparator.comparingInt(Cursor::value)
                        .thenComparingInt(Cursor::source)
                        .thenComparingInt(Cursor::index));
        int total = 0;
        for (int source = 0; source < sources.length; source++) {
            int[] row = sources[source];
            if (row == null) {
                throw new IllegalArgumentException("source must not be null");
            }
            total = Math.addExact(total, row.length);
            for (int i = 1; i < row.length; i++) {
                if (row[i] < row[i - 1]) {
                    throw new IllegalArgumentException("sources must be sorted");
                }
            }
            if (row.length > 0) {
```

JAVA (continued 4/7)

```
                frontier.add(new Cursor(row[0], source, 0));
            }
        }
        List<Integer> result = new ArrayList<>(total);
        while (!frontier.isEmpty()) {
            Cursor next = frontier.remove();
            result.add(next.value());
            int following = next.index() + 1;
            if (following < sources[next.source()].length) {
                frontier.add(new Cursor(sources[next.source()][following],
                        next.source(), following));
            }
        }
        return result;
    }

    public static int minimumMeetingRooms(int[][] intervals) {
        if (intervals == null) {
            throw new IllegalArgumentException("intervals must not be null");
        }
        int[][] copy = new int[intervals.length][2];
        for (int i = 0; i < intervals.length; i++) {
            if (intervals[i] == null || intervals[i].length != 2
                    || intervals[i][0] > intervals[i][1]) {
                throw new IllegalArgumentException("invalid interval");
            }
            copy[i] = intervals[i].clone();
        }
        Arrays.sort(copy, Comparator.comparingInt((int[] interval) -> interval[0])
                .thenComparingInt(interval -> interval[1]));
        PriorityQueue<Integer> activeEnds = new PriorityQueue<>();
        int maximum = 0;
        for (int[] interval : copy) {
```

JAVA (continued 5/7)

```
            if (interval[0] == interval[1]) {
                continue;
            }
            while (!activeEnds.isEmpty() && activeEnds.element() <= interval[0]) {
                activeEnds.remove();
            }
            activeEnds.add(interval[1]);
            maximum = Math.max(maximum, activeEnds.size());
        }
        return maximum;
    }

    public static final class RunningMedian {
        private final PriorityQueue<Integer> lower =
                new PriorityQueue<>(Comparator.reverseOrder());
        private final PriorityQueue<Integer> upper = new PriorityQueue<>();

        public void add(int value) {
            if (lower.isEmpty() || value <= lower.element()) {
                lower.add(value);
            } else {
                upper.add(value);
            }
            if (lower.size() < upper.size()) {
                lower.add(upper.remove());
            } else if (lower.size() > upper.size() + 1) {
                upper.add(lower.remove());
            }
        }

        public double median() {
            if (lower.isEmpty()) {
```

JAVA (continued 6/7)

```
                throw new IllegalStateException("no values");
            }
            if (lower.size() > upper.size()) {
                return lower.element();
            }
            return ((long) lower.element() + upper.element()) / 2.0;
        }

        public int size() {
            return lower.size() + upper.size();
        }
    }

    private static void siftDownMax(int[] values, int root, int size) {
        int current = root;
        while (true) {
            int left = 2 * current + 1;
            if (left >= size) {
                return;
            }
            int right = left + 1;
            int larger = right < size && values[right] > values[left] ? right : left;
            if (values[current] >= values[larger]) {
                return;
            }
            int temporary = values[current];
            values[current] = values[larger];
            values[larger] = temporary;
            current = larger;
        }
    }
```

JAVA (continued 7/7)

```
    private static void requireArray(int[] values) {
        if (values == null) {
            throw new IllegalArgumentException("values must not be null");
        }
    }

    public static void main(String[] args) {
        int[] heap = {3, 1, 6, 5, 2, 4};
        heapifyMax(heap);
        assert isMaxHeap(heap);
        assert heap[0] == 6;
        assert kthLargest(new int[] {3, 2, 1, 5, 6, 4}, 2) == 5;
        assert topKFrequent(new int[] {1, 1, 1, 2, 2, 3}, 2)
                .equals(List.of(1, 2));

        assert mergeSorted(new int[][] {{1, 4, 9}, {2, 2, 8}, {3, 7}})
                .equals(List.of(1, 2, 2, 3, 4, 7, 8, 9));
        assert minimumMeetingRooms(new int[][] {{0, 30}, {5, 10}, {15, 20}}) == 2;
        assert minimumMeetingRooms(new int[][] {{5, 5}}) == 0;

        RunningMedian median = new RunningMedian();
        median.add(5);
        assert median.median() == 5.0;
        median.add(2);
        assert median.median() == 3.5;
        median.add(10);
        assert median.median() == 5.0;
        median.add(4);
        assert median.median() == 4.5;
        assert median.size() == 4;
    }
}
```

## Complexity and failure-mode table

| Operation/pattern | Time | Space | Main risk |
|---|---|---|---|
| bottom-up heapify | `O(n)` | `O(1)` | treating every node as height `log n` |
| kth largest | `O(n log k)` | `O(k)` | wrong min/max heap orientation |
| top-k frequency | expected `O(n + d log k)` | `O(d + k)` | undefined tie order |
| k-way merge | `O(N log k)` | `O(k)` plus output | storing all elements in the heap |
| streaming median insertion | `O(log n)` | `O(n)` | order or balance invariant broken |
| meeting rooms | `O(n log n)` | `O(n)` | endpoint reuse semantics undefined |
| lazy deletion update | amortized `O(log n)` | can exceed live size | stale metadata never compacted |
| quickselect | expected `O(n)`, worst `O(n^2)` | depends on implementation | mutation and pivot worst case |

## WATCH OUT | Edge cases and common mistakes

1. **Wrong heap orientation.** Kth largest retains a min-heap so the weakest retained value is removable.
2. **Overflowing comparator.** Never implement priority with subtraction.
3. **Mutable priority field.** Reinsert an immutable updated entry.
4. **Assuming iteration is sorted.** Only repeated `poll` produces priority order and it destroys the queue.
5. **Invalid k.** Define `k=0` only for top-k output; kth rank requires `1..n`.
6. **Dropping duplicates.** Rank by occurrences unless the question explicitly says distinct values.
7. **Median overflow.** Widen before adding two middle integers.
8. **One heap unbalanced.** State which side owns the odd extra value.
9. **K-way merge without sorted validation.** The frontier proof depends on every source order.
10. **Closed versus half-open meetings.** Decide whether an end at time `t` frees a room for a start at `t`; under `[start,end)`, `[t,t)` is empty and must not allocate a room.
11. **Arbitrary queue removal assumed logarithmic.** Standard `remove(Object)` searches linearly.
12. **Ignoring output space.** Merging into a list stores all `N` results even though the frontier is `O(k)`.

## SDE-2 production follow-ups

- **Backpressure:** a k-way streaming merge must not read unbounded data ahead of a slow consumer.
- **Failure policy:** decide whether one failed source aborts the merge, is retried, or yields a partial result with provenance.
- **Concurrency:** `PriorityQueue` is not thread-safe. `PriorityBlockingQueue` is concurrent but unbounded by default and does not supply every scheduling policy.
- **Capacity:** bounded top-k heaps naturally cap candidate memory, but the preceding frequency map may still have unbounded distinct cardinality.
- **Determinism:** include stable tie-break fields such as sequence number when equal priority must preserve arrival order.
- **Clock semantics:** schedulers need monotonic time for durations, wall time for calendar meaning, and a policy for clock changes.
- **Cancellation:** stable job IDs plus lazy deletion or an indexed heap avoid repeated linear scans.
- **Metrics:** track live size, physical size, delayed-delete count, oldest wait, processing lag, and rejected jobs.
- **Numeric types:** weights, timestamps, sums, and counts often require `long`; median averaging may require `double` or an exact rational/decimal contract.
- **Library choice:** prefer Java's maintained queue for object priorities. A custom primitive heap is justified by measured allocation or API needs and deserves dedicated property tests.

## TRY IT | Exercises with model checkpoints

### Exercise 1: kth smallest in sorted rows

Find the kth smallest value across sorted arrays without flattening.

**Model checkpoints:** use the k-way frontier; pop exactly k times; validate total count and k; complexity `O(k log rows)` and space `O(rows)`; duplicates count as separate occurrences.

### Exercise 2: sliding-window median

Return the median for every width-k window.

**Model checkpoints:** two heaps plus delayed deletion; identify entries by `(value,index)` to disambiguate duplicates; maintain logical sizes; prune before reading roots; `O(n log k)` expected/amortized and `O(k)` live state, with possible stale physical entries.

### Exercise 3: assign meeting-room IDs

Return a room number for each meeting.

**Model checkpoints:** active heap ordered by end time and a second min-heap of free room IDs; retain original interval index; define tie reuse; output assignments while maximum allocated ID gives room count.

### Exercise 4: merge infinite iterators

Merge sorted iterators lazily.

**Model checkpoints:** one frontier element per source; do not call `next` without `hasNext`; surface source failures; close resources; consumer cancellation must stop producers; returned iterator owns the heap.

### Exercise 5: top-k with changing scores

Items receive score updates and queries request current top k.

**Model checkpoints:** inserting new snapshots creates stale entries; version each item and lazily discard old roots, or use an indexed heap; cap stale growth; define tie order and consistency during concurrent updates.

### Exercise 6: choose selection strategy

Compare sorting, quickselect, and a bounded heap for 100 million values and `k=20`.

**Model checkpoints:** streaming and memory favor the heap; offline mutable storage may favor quickselect; sorted downstream consumption favors sorting; include I/O cost, parallelism, worst-case requirement, and repeated-query plans.

## Interview answer checklist

- **[ ]** I stated the heap orientation and comparator tie order.
- **[ ]** I named the retained-candidate or frontier invariant.
- **[ ]** I used expected/amortized wording where appropriate.
- **[ ]** I counted duplicates according to the rank contract.
- **[ ]** I widened comparator and median arithmetic safely.
- **[ ]** I separated live state, delayed stale state, and output space.
- **[ ]** I defined empty, k-boundary, and interval-endpoint behavior.
- **[ ]** I can compare heap, sorting, buckets, and quickselect.
- **[ ]** I can explain cancellation, backpressure, determinism, and concurrency.

## RECAP | Summary

Heaps maintain an extreme under updates, not a complete order. Bottom-up heapify derives a valid tree in linear time. A bounded min-heap retains the largest k values; a map plus heap retains top frequencies; a frontier heap merges sorted sources; two ordered heaps expose a streaming median; and end-time heaps model active scheduled work. Lazy deletion compensates for arbitrary-removal limits, while quickselect offers a different offline rank trade-off. A production-grade answer makes the comparator, invariant, tie policy, retained memory, stale state, and operational behavior explicit.

Part II - Practice and Handoff

# Practice Ladder and Next Steps

## TRY IT | Practice ladder

- Foundation: heap operations and kth values
- Interview Core: top-k and k-way merge
- SDE-2 Follow-up: comparator and memory trade-offs
- Challenge: streaming median or scheduler design

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

### Navigation

Previous volume: 13 - Trees, Binary Search Trees, and Tries

Series index

Next volume: 15 - Graphs: Traversal, Ordering, Paths, and Union-Find

## TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

# Series Roadmap

Complete the learning steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. Stable volume numbers remain in filenames; the steps below show the recommended reading order. Click a title when your PDF viewer permits local sibling-file links.

| STEP | FOCUSED LEARNING PATH |
|---|---|
| 1 | Java Foundations for Problem Solving |
| 2 | Time and Space Complexity |
| 3 | Number Systems and Math Foundations for DSA Interviews |
| 4 | Bit Manipulation in Java |
| 5 | Loop Mastery, Patterns, and Index Calculations |
| 6 | Arrays and Array Problem-Solving Patterns |
| 7 | Strings and String Problem-Solving Patterns |
| 8 | Hashing: Maps, Sets, Frequency, and Prefix State |
| 9 | Recursion and Backtracking in Java |
| 10 | Linked Lists: Pointer Reasoning and Mutation |
| 11 | Stacks, Queues, Deques, and Monotonic Patterns |
| 12 | Binary Search: Bounds, Answers, and Invariants |
| 13 | Trees, Binary Search Trees, and Tries |
| 14 | Heaps, Priority Queues, Selection, and Top-K |
| 15 | Graphs: Traversal, Ordering, Paths, and Union-Find |
| 16 | Greedy Algorithms: Recognition, Proofs, and Scheduling |
| 17 | Dynamic Programming: State, Transitions, and Optimization |
| 18 | Advanced Java Engineering and the SDE-2 Interview (Parts A-J) |

Learning Step 3 uses a linked Number Systems foundations volume and workbook; the final advanced stage uses a ten-part collection, including a three-volume backend specialist track. These splits keep every file focused and printable.

# About the Author

## Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

## Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

## Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

## Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

**Connect:** LinkedIn | GitHub

# Copyright and Publishing Notes



Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Volume 14 of 18. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.